

Loss Function

(Cross Entropy)

& 연구 주제

➤ 이진 분류(Binary Classification) 와 CE

$$\bullet H(P, Q) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] = -(p_1 \log q_1 + p_2 \log q_2)$$

(1) 실제 값 $y = 1$ (즉, $p_1 = 1$)

만약 $\hat{y} = 1$ 이면 (즉, $q_1 = 1$), 완벽한 예측 $\rightarrow$ 손실 $H = 0$ (최소값)

만약 $\hat{y} = 0.5$ 이면, 예측이 불확실 $\rightarrow$ 손실 증가

만약 $\hat{y} = 0$ 이면 (완전히 틀림), $-\log(0) = \infty \rightarrow$ 손실 무한대!

- $p_1 = 1$ 이므로 q_1 도 커야 좋은 예측 (즉, p_1 가 크면 q_1 도 커야 함)

(2) 실제 값 $y = 0$ (즉, $p_1 = 0$)

만약 $\hat{y} = 0$ 이면 (즉, $q_1 = 0$), 완벽한 예측 $\rightarrow$ 손실 $H = 0$ (최소값)

만약 $\hat{y} = 0.5$ 이면, 예측이 불확실 $\rightarrow$ 손실 증가

만약 $\hat{y} = 1$ 이면 (완전히 틀림), $-\log(1 - 1) = \infty \rightarrow$ 손실 무한대!

- $p_1 = 0$ 이므로 q_1 도 작아야 좋은 예측 (즉, p_1 가 작으면 q_1 도 작아야 함)

➤ 다중 클래스 분류(Multi-class Classification) 와 CE

- 다중 클래스 분류에서 크로스 엔

p_i 가 크면 q_i 도 커야 함 → 해당 클래스의 예측 확률이 높아야 함

p_i 가 작으면 q_i 도 작아야 함 → 해당 클래스의 예측 확률이 낮아야 함

즉, q_i 는 p_i 와 비슷해야 좋은 예측

예)

- class 4 개인 경우 정답이 고양이 (class = 3) 라고 할 때:

정답 분포: $y = [0,0,1,0]$

1) 모델 예측: $\hat{y} = [0.1,0.2,0.6,0.1]$

손실값: $L = -\log(0.6) \approx 0.51$

2) 만약 모델이 틀리게 예측했다면 (예: $\hat{y} = [0.7,0.2,0.05,0.05]$):

$$L = -\log(0.05) \approx 2.99$$

즉, 잘못된 예측일수록 loss가 커짐.

➤ Cross-Entropy (CE) 수식의 단점

1. Outlier(이상치)에 민감함

Cross-Entropy는 정답 클래스의 예측 확률이 아주 작으면 $-\log(\hat{y})$ 값이 폭발적으로 커짐.

예: 정답 확률을 0.001 로 예측하면 손실 = $-\log(0.001) \approx 6.9$

따라서 데이터 노이즈나 잘못된 라벨이 있으면 전체 학습에 큰 영향을 줌.

2. Imbalanced Data(불균형 데이터) 문제

Cross-Entropy는 클래스 간 비율을 고려하지 않음.

다수 클래스가 많을 경우 손실값이 대부분 다수 클래스 기준으로 학습됨

→ 소수 클래스는 무시될 수 있음.

3. Overconfidence 문제

- 모델이 틀린 예측을 "아주 확신"할 때 손실이 극단적으로 커짐.
- 예: 정답은 "고양이"인데 모델이 "개"에 대해 0.99 확률을 준 경우
→ 큰 패널티.
- 결과적으로 **softmax** 출력이 과도하게 **sharp**해지고, **overfitting** 위험이 있음.

•연구 주제

➤ Outlier 에 민감한 문제를 해결하기 위한 Loss function - 기존방법

- **Huber Loss (후버 손실):** 회귀(regression) 문제에서 많이 사용되는 후버 손실은 작은 오차에 대해선 MSE(제곱 오차)처럼, 큰 오차에 대해선 MAE(절대 오차)처럼 작동. 이는 큰 오차(아웃라이어)에 의한 패널티가 무한정 커지지 않도록 손실 함수의 기울기를 완화하는 방식.
- **Generalized Cross-Entropy (GCE):** GCE는 CE 손실과 MAE(Mean Absolute Error) 손실을 결합한 형태로, 아웃라이어에 덜 민감하도록 설계. 이는 CE의 $-\log(y^i)$ 항이 MAE의 선형적인 형태로 변형되도록 하여 손실 값의 증가를 억제함.
- **Focal Loss (포컬 손실):** 포컬 손실은 클래스 불균형 문제를 해결하기 위해 고안되었지만, 그 원리도 '잘 분류된 쉬운 샘플(easy examples)'에 대한 손실을 감소시켜 모델이 '분류하기 어려운 샘플(hard examples)'에 집중하도록 유도한다는 점에서 아웃라이어에 대한 영향을 줄이는 효과를 가짐

등등.... (기존 연구 논문들 찾아서 아이디어 파악하고 수식정리하기)

Capped Cross-Entropy (CCE)

$$L(p_t) = -\log(\max(q_t, \varepsilon)), 0 < \varepsilon < 1$$

- 장점: 간단, 직관적, outlier 억제.
- 적용: outlier , 불균형(imbalanced) 데이터, 라벨 노이즈 많은 상황에서 안정적.

Soft-Capped Cross-Entropy (SCCE)

$$L_{\text{SCCE}}(p_t) = -\log(p_t + \varepsilon), \quad 0 < \varepsilon < 1$$

- 기존 CCE 확률에 작은 ε 을 더함 $\rightarrow$ "부드러운 바닥" 역할.

ε -Smoothed Cross-Entropy (ε -CE)

- $L_\varepsilon(p_t) = -\log(\varepsilon + (1 - \varepsilon)q_t), 0 < \varepsilon < 1$

➤ Imbalanced Data 문제를 해결하기 위한 Loss function - 기존방법

- **Weighted Cross-Entropy (WCE):** 클래스별 샘플 수에 반비례하는 가중치를 곱해 손실을 계산하여, 소수 클래스의 영향력을 강화하는 방법
- **Balanced Cross-Entropy (BCE):** 이진 분류에서 양성과 음성 클래스가 데이터 비율에 상관없이 동일하게 학습되도록 각 클래스에 비중을 조정한 손실함수
- **Focal Loss:** 쉽게 맞추는 샘플의 손실 기여도를 줄이고, 어려운 샘플이나 소수 클래스에 집중할 수 있도록 만든 손실 함수
- **Class-Balanced Loss:** 단순히 샘플 수가 아니라 실질적인 샘플 수(effective number of samples)라는 개념을 도입해 클래스별 가중치를 안정적으로 조정한 손실함수

등등 (기존 연구 논문들 찾아서 아이디어 파악하고 수식정리하기)

Adaptive Class-Weighted Cross Entropy Loss

$$\bullet L(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{i=1}^c w_i \cdot y_i \log(\hat{y}_i),$$

$$w_i = \frac{1}{\log(1 + n_i)}$$

- 클래스 불균형 문제를 해결하기 위해 각 클래스에 가중치를 부여
- w_i 는 클래스 i 의 가중치로, 학습 과정에서 adaptive 하게 업데이트 됨, n_i 는 클래스 i 의 샘플 수
- 이 방법은 클래스 불균형 문제를 해결하면서도, 학습 과정에서 각 클래스의 중요도를 동적으로 반영 가능.

➤ 추천 머신러닝 기법

- Logistic reg (+ multi-logistic reg) / Soft max reg
- Naïve Bayes (log-likelihood 를 쓰지만 CE 사용하기도 함)
- 의사결정나무 & 랜덤 포레스트 (Decision Tree, Random Forest)
(Gini index나 엔트로피(entropy)를 쓰지만, CE 사용가능)
- XGBoost, LightGBM, CatBoost

- 머신러닝 Original Loss vs Cross-Entropy(CE) 비교

1. 로지스틱 회귀 (Logistic Regression)

- Original loss function:

log-likelihood의 음수, 즉 CE와 동일

$$L_{\text{orig}} = - \sum_{i=1}^N [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

- CE 손실:

$$L_{\text{CE}} = L_{\text{orig}}$$

⇒ 차이 없음 (로지스틱 회귀는 CE가 기본 손실).

2. Softmax 회귀 (Multinomial Logistic Regression)

- Original loss function:

다항 로지스틱 log-likelihood = CE

$$L_{\text{orig}} = - \sum_{i=1}^N \sum_{k=1}^K y_{i,k} \log p(y = k \mid x_i)$$

- CE 손실:

$$L_{\text{CE}} = L_{\text{orig}}$$

⇒ 동일.

3. 나이브 베이즈 (Naive Bayes)

- Original loss function:

log-likelihood 최대화

$$L_{\text{orig}} = - \sum_{i=1}^N \log p(y_i | x_i)$$

- CE 손실:

$$L_{\text{CE}} = - \sum_{i=1}^N \sum_{k=1}^K y_{i,k} \log p(y = k | x_i)$$

⇒ 사실상 동일 (one-hot 레이블을 사용하면 MLE = CE).

4. Decision Tree (CART 기준)

- Original loss function (노드 불순도):
Gini index 또는 엔트로피(Information Gain)

- Gini:

$$G(t) = \sum_{k=1}^K p_k(1 - p_k)$$

- Entropy:

$$H(t) = - \sum_{k=1}^K p_k \log p_k$$

- CE 손실 (leaf에서 예측 확률 분포 기반):

$$L_{CE} = - \sum_{i=1}^N \sum_{k=1}^K y_{i,k} \log p(y = k | x_i)$$

⇒ 트리 학습 과정에서는 Gini/Entropy 사용, 학습 완료 후 예측 성능 측정에서는 CE를 사용 가능.

5. Random Forest

- Original loss function:

개별 트리에서 Gini/Entropy 사용

- CE 손실: 앙상블 확률 평균 후 CE로 평가

$$L_{\text{CE}} = - \sum_{i=1}^N \sum_{k=1}^K y_{i,k} \log \left(\frac{1}{M} \sum_{m=1}^M p_m(y = k \mid x_i) \right)$$

6. Gradient Boosting (예: XGBoost, LightGBM)

- Original loss function:

선택 가능 (기본은 log-loss, 즉 CE)

- 이진 분류:

$$L_{\text{orig}} = - \sum_{i=1}^N [y_i \log p_i + (1 - y_i) \log (1 - p_i)]$$

- 회귀: MSE 사용 가능

- CE 손실: 분류 태스크에서 기본 손실 함수가 CE

$$L_{\text{CE}} = L_{\text{orig}}$$

➤ 연구순서

- 1) 기존연구 찾기 (논문찾고, 어떤 방법 썼는지 조사하기)
- 2) 어떤 ML 방법쓸지 결정하기
- 3) 데이터 찾기 (UCI data repository, Kaggle 등 / 민경, 지아, 하랑한테 도움 받기)
- 4) 데이터 분석 (코딩, 기존방법과 우리 방법 비교하기)